Sorting of Data

06 June 2026 08:12

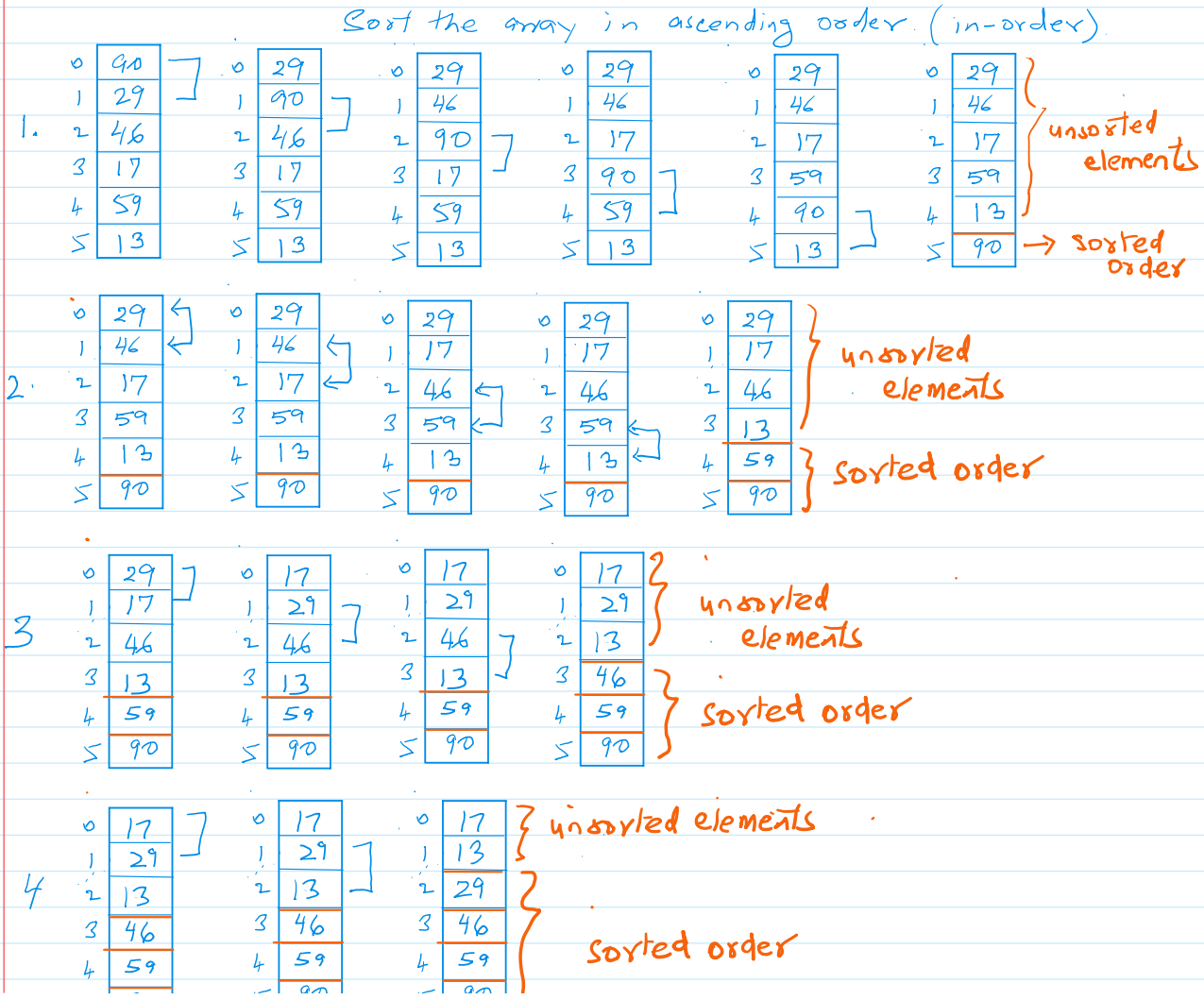
Sorting means arrangement of data either in ascending order or in descending order.

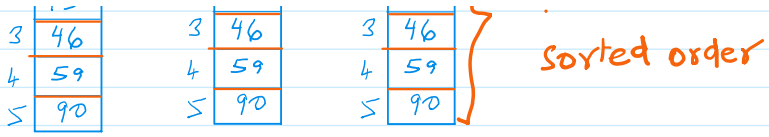
Why we do Sorting?
The benefit of Sorting, it makes searching data easier and faster.

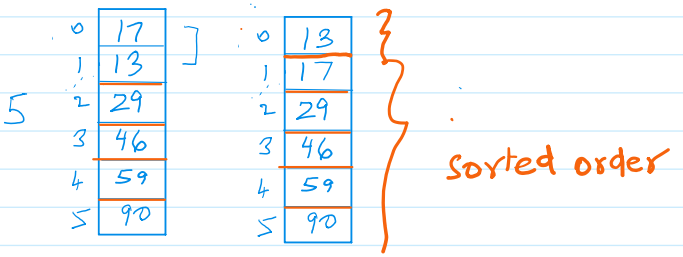
Fundamental sorting techniques

1. **Bubble Sort**
2. **Selection Sort**
3. **Insertion Sort (Not in ICSE)**

1. **Bubble Sort:** Bubble sort is one of the simplest sorting algorithms. It repeatedly compares adjacent elements and swaps them if they are in the wrong order. This process continues until the array is sorted.
   The unsorted elements bubbles up and sorted elements sediments at its correct position after each pass, which is why it is called bubble sort.







For 6 elements we need 5 passes.
For 7 " " " 6 passes
:
For n " " " (n-1) passes

$$S_N = \frac{N(N+1)}{2}$$

$$= \frac{5(5+1)}{2}$$

$$= \frac{5 \times 6^3}{2}$$

$$= 15$$

$5+4+3+2+1 = 15$

For 6 elements we need 15 comparisons.
For 7 elements we need (15+6) = 21 comparisons.
For 8 elements we need (21+7) = 28 comparisons.
:
n " " " $S_N = \frac{N(N+1)}{2} =$

put $N = n-1$

$$S_n = \frac{(n-1)(n-1+1)}{2} = \frac{n(n-1)}{2}$$

Number of comparisons in bubble sort = $\frac{n(n-1)}{2}$

$$= \frac{n^2}{2} - \frac{n}{2}$$

↑ dominating term.

$f(x) = x^2 + 1$

$n = 10$

$n^2 = 100$

$n = 100$

$n^2 = 10000$

$n = 1000 = 10^3$

$n^2 = 10^6$

We define a function for dominating term, we call it-

Big-O = $O(n^2)$

↑

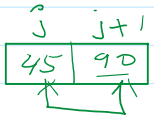
In Big-O function we won't write constants or non-dominating terms.

Big-O function represent the worst case scenario for an algorithm.

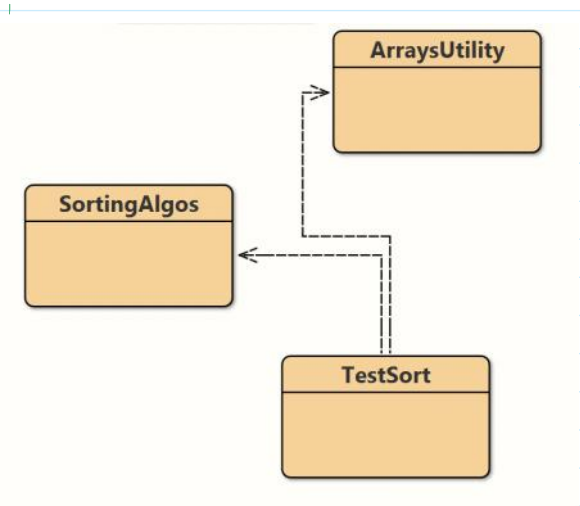
Data (Array)	Number of operations (n^2)
10	100
20	400

10	100
20	400
30	900
40	1600
100	10000
1000	10^6

```
public void bubbleSort(int a[]){
    int i, j, t;
    for(i=0; i<a.length-1; i++){
        for(j=0; j<a.length-1-i; j++){
            if(a[j] > a[j+1]){
                t = a[j];
                a[j] = a[j+1];
                a[j+1] = t;
            }
        }
    }
}
```



t 90




```
package MyArrays;

/**
 * Write a description of class SortingAlgos here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class SortingAlgos
{
```

```
    public static void bubblesort (int a[]) {
        int i, j, t;
        for (i=0;i<a.length-1; i++) {
            for (j =0; j<a.length-1-i; j++){
                if(a[j]> a[j+1]) {
                    t = a[j];
                    a[j] = a[j+1];
                    a[j+1]=t;
                }
            }
        }
    }
}

package MyArrays;
import java.util.Scanner;

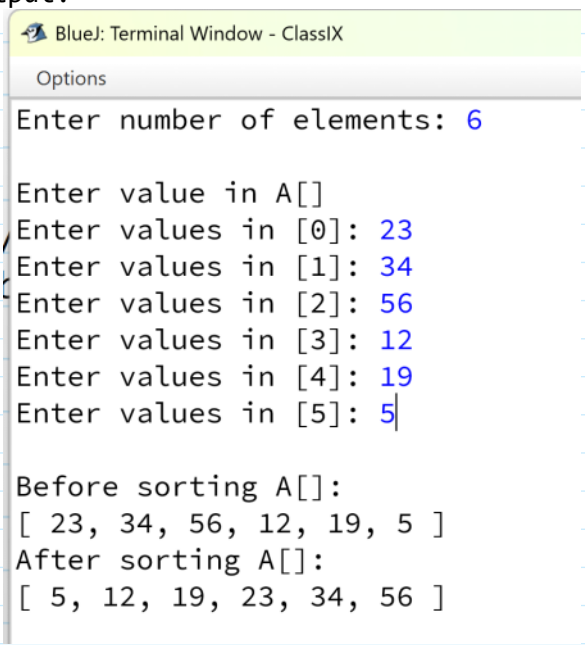

/**
 * Write a description of class ArraysUtility here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ArraysUtility
{
    private static Scanner sc = new Scanner(System.in);
    public static void input(int A[], String msg){
        System.out.print("\n" + msg + "\n");
        int i;
        for(i = 0; i < A.length; i++){
            System.out.print("Enter values in ["+i+"]: ");
            A[i] = sc.nextInt();
        }
    }
    public static void printArray(int A[], String msg){
        System.out.print("\n" + msg + "\n[ ");
        int i;
        for(i = 0; i < A.length - 1; i++){
            System.out.print(A[i]+", ");
        }
        System.out.print(A[i]+" ]");
    }
}

package MyArrays;
import java.util.*;


/**
 * Write a description of class TestSort here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestSort
```

```
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int numOfElements, A[], i;
        System.out.print("\fEnter number of elements: ");
        numOfElements = sc.nextInt();
        A = new int[numOfElements];
        ArraysUtility.input(A, "Enter value in A[]");
        ArraysUtility.printArray(A, "Before sorting A[]:");
        SortingAlgos.bubblesort(A);
        ArraysUtility.printArray(A, "After sorting A[]:");
    }
}
```

Output:



School technique to write code:

```
package MyArrays;
import java.util.*;

/**
 * Write a description of class SchoolTecnique here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class SchoolTecnique
{
    private static Scanner sc = new Scanner(System.in);
    public static void bubblesort (int a[]) {
        int i, j, t;
        for (i=0;i<a.length-1; i++) {
            for (j =0; j<a.length-1-i; j++){
                if(a[j]> a[j+1]) {
                    t = a[j];
                    a[j] = a[j+1];
                    a[j+1]=t;
                }
```

```
            }
        }
    }

    public static void input(int A[], String msg){
        System.out.print("\n" + msg + "\n");
        int i;
        for(i = 0; i < A.length; i++){
            System.out.print("Enter values in ["+i+"]: ");
            A[i] = sc.nextInt();
        }
    }

    public static void printArray(int A[], String msg){
        System.out.print("\n" + msg + "\n[ ");
        int i;
        for(i = 0; i < A.length - 1; i++){
            System.out.print(A[i]+", ");
        }
        System.out.print(A[i]+" ]");
    }

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int numOfElements, A[], i;
        System.out.print("\fEnter number of elements: ");
        numOfElements = sc.nextInt();
        A = new int[numOfElements];
        input(A, "Enter value in A[]");
        printArray(A, "Before sorting A[]:");
        bubblesort(A);
        printArray(A, "After sorting A[]:");
    }
}
```

Output will be same.

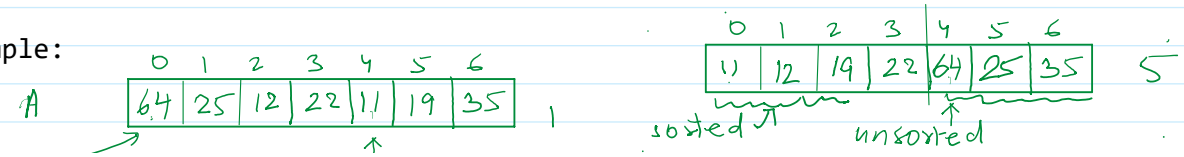
Selection Sort

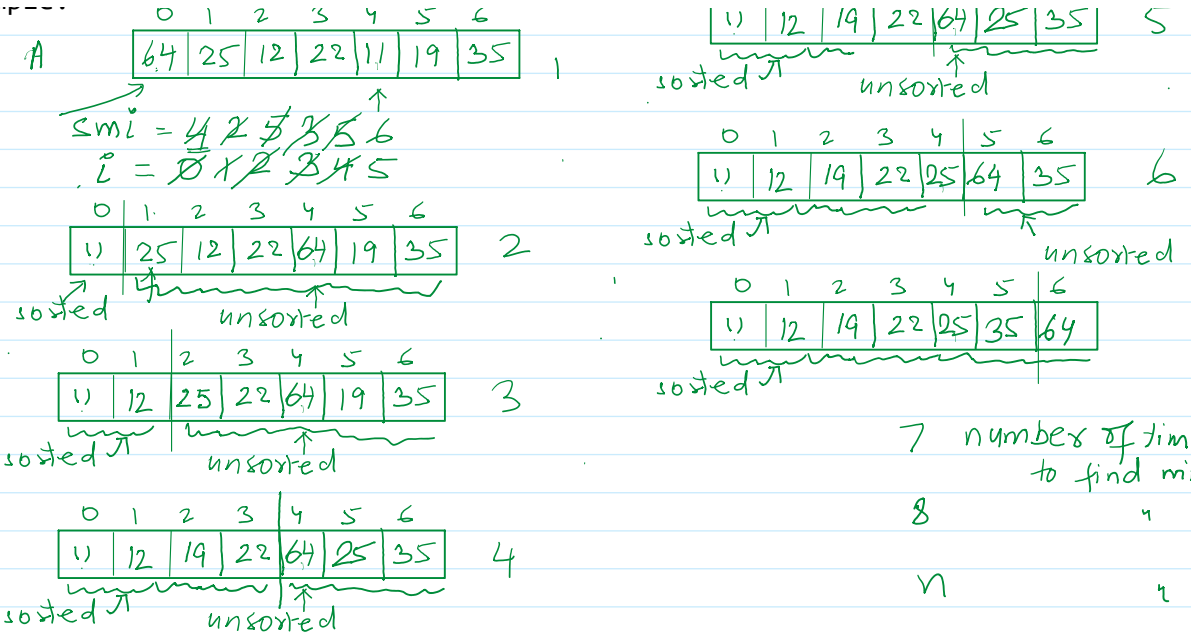
Selection Sort is a simple comparison-based sorting algorithm. It repeatedly finds the smallest element from the unsorted part of the array and places it at the correct position in the sorted part.

Basic idea

1. Find the smallest element in the array.
2. Swap it with the first element.
3. Find the next smallest element from the remaining array.
4. Swap it with the second element.
5. Repeat until the entire array is sorted.

Example:





7 number of times we need to find min index = 6

8 " " = 7

n " " = n-1

$= \frac{n \times (n-1)}{2} = O(n^2)$ performance

Performance of selection sort: $O(n^2)$

```
public static void selectionSort(int a[]){
        int i, j, minIndex, t;
        for(i = 0; i < a.length - 1; i++){
            //loop to find smallest element index
            for(minIndex = i, j = i+1; j < a.length; j++){
                if(a[minIndex] > a[j]){
                    minIndex = j;
                }
            }
            //swap the ith index value with minIndex value
            t = a[i];
            a[i] = a[minIndex];
            a[minIndex] = t;
        }
    }
```

Output:

```
BlueJ: Terminal Window - ClassIX
Options
Enter value in A[]
Enter values in [0]: 71
Enter values in [1]: 52
Enter values in [2]: 39
Enter values in [3]: 8
Enter values in [4]: 19
Enter values in [5]: 22
Enter values in [6]: 33
Enter values in [7]: 90
Enter values in [8]: 16
Enter values in [9]: 5

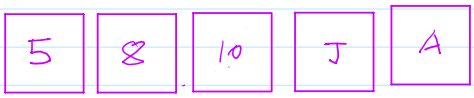
Before sorting A[]:
[ 71, 52, 39, 8, 19, 22, 33, 90, 16, 5 ]
After sorting A[]:
[ 5, 8, 16, 19, 22, 33, 39, 52, 71, 90 ]
Can only enter input while your program i
```

Function that sorts the same array according to users choice.

```
/**
     * @param a int[]
     * @param isAsc if value is true then it arranges the array
     * in ascending order otherwise in descending order
     */
    public static void selectionSort(int a[], boolean isAsc){
        int i, j, index, t;
        for(i = 0; i < a.length - 1; i++){
            //loop to find smallest element index
            for(index = i, j = i+1; j < a.length; j++){
                if(a[index] > a[j] && isAsc){
                    index = j;
                }
                else if(a[index] < a[j] && !isAsc){
                    index = j;
                }
            }
            //swap the ith index value with minIndex value
            t = a[i];
            a[i] = a[index];
            a[index] = t;
        }
    }
```

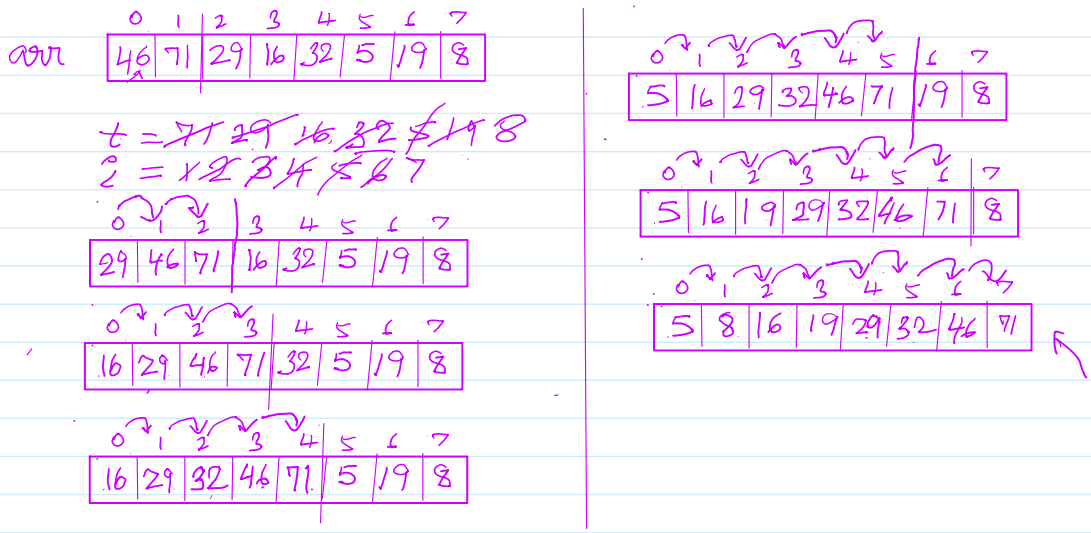
Insertion Sort

14 13 12 11
2, 3, 4, 5, 6, 7, 8 9, 10, A, K, 8, J



Insertion sort is a simple sorting algorithm that works similarly how we arrange playing cards in our hand.

It builds the sorted array **one element at a time** by inserting each element into its correct position in the already sorted part of the array



Algorithm.

1. Start
2. Assume first element is already sorted.
3. Pick next element (key).
4. Compare key with previous elements.
5. Shift larger elements one position right.
6. Insert key in correct position.
7. Repeat until all elements are processed.
8. Stop.

```
public static void insertionSort(int a[]){
        int i, j, key;
        for(i = 1; i < a.length; i++){
            for(key = a[i], j = i - 1; j >= 0 && key < a[j]; a[j+1] = a[j],j--);
            //{
                //a[j+1] = a[j]; //shifting of previous element to next index
            //}
            a[j+1] = key;
        }
    }
```

Sorting the array in both ways

```
/**
     * @param a int[]
```

```
     * @param isAsc if value is true then it arranges the array
     * in ascending order otherwise in descending order
     */

    public static void insertionSort(int a[], boolean isAsc){
        int i, j, key;
        for(i = 1; i < a.length; i++){
            //loop for ascending
            for(key = a[i], j = i - 1; isAsc && j >= 0 && key < a[j]; a[j+1] =
a[j],j--);
            //loop for descending
            for(; !isAsc && j >= 0 && key > a[j]; a[j+1] = a[j],j--);
            a[j+1] = key;
        }
    }
```

```
BlueJ: Terminal Window - ClassIX
Options
Enter number of elements: 5

Enter value in A[]
Enter values in [0]: 71
Enter values in [1]: 92
Enter values in [2]: 45
Enter values in [3]: 109
Enter values in [4]: 56

Before sorting A[]:
[ 71, 92, 45, 109, 56 ]
After sorting A[]:
[ 109, 92, 71, 56, 45 ]
```

descending order

```
BlueJ: Terminal Window - ClassIX
Options
Enter number of elements: 5

Enter value in A[]
Enter values in [0]: 61
Enter values in [1]: 432
Enter values in [2]: 12
Enter values in [3]: 16
Enter values in [4]: 19

Before sorting A[]:
[ 61, 432, 12, 16, 19 ]
After sorting A[]:
[ 12, 16, 19, 61, 432 ]
```

ascending order